# Cadernos Jupyter com Geopandas e Leafmap

## Disciplina: Programação aplicada à engenharia cartográfica

Maurício C. M. de Paulo - D.Sc.

1 de abril de 2026

# Sumário


1. Cadernos jupyter
   - Google Colab
   - Jupyter lab
   - VSCodium Jupyter plugin
2. Estruturas de dados avançadas no Jupyter
3. Visualização de geoinformação no Jupyter

Cadernos Jupyter com Geopandas e Leafmap

# Cadernos jupyter

# Jupyter Notebooks — Conceito

Jupyter Notebook é um ambiente interativo para:

- Executar código (Python, R, etc.)
- Visualizar resultados imediatamente
- Combinar código, texto e gráficos

**Muito usado em:**

- Ciência de dados
- Geoprocessamento
- Ensino e prototipagem

Existem várias formas de usar:

- Google Colab
- VSCode / VSCodium
- JupyterLab local

# Google Colab (na nuvem)

Executa notebooks diretamente no navegador.

**Como usar:**

- Acessar: `https://colab.research.google.com`
- Login com conta Google
- Criar novo notebook

**Instalação de pacotes**

```
!pip install geopandas
```

**Vantagens**

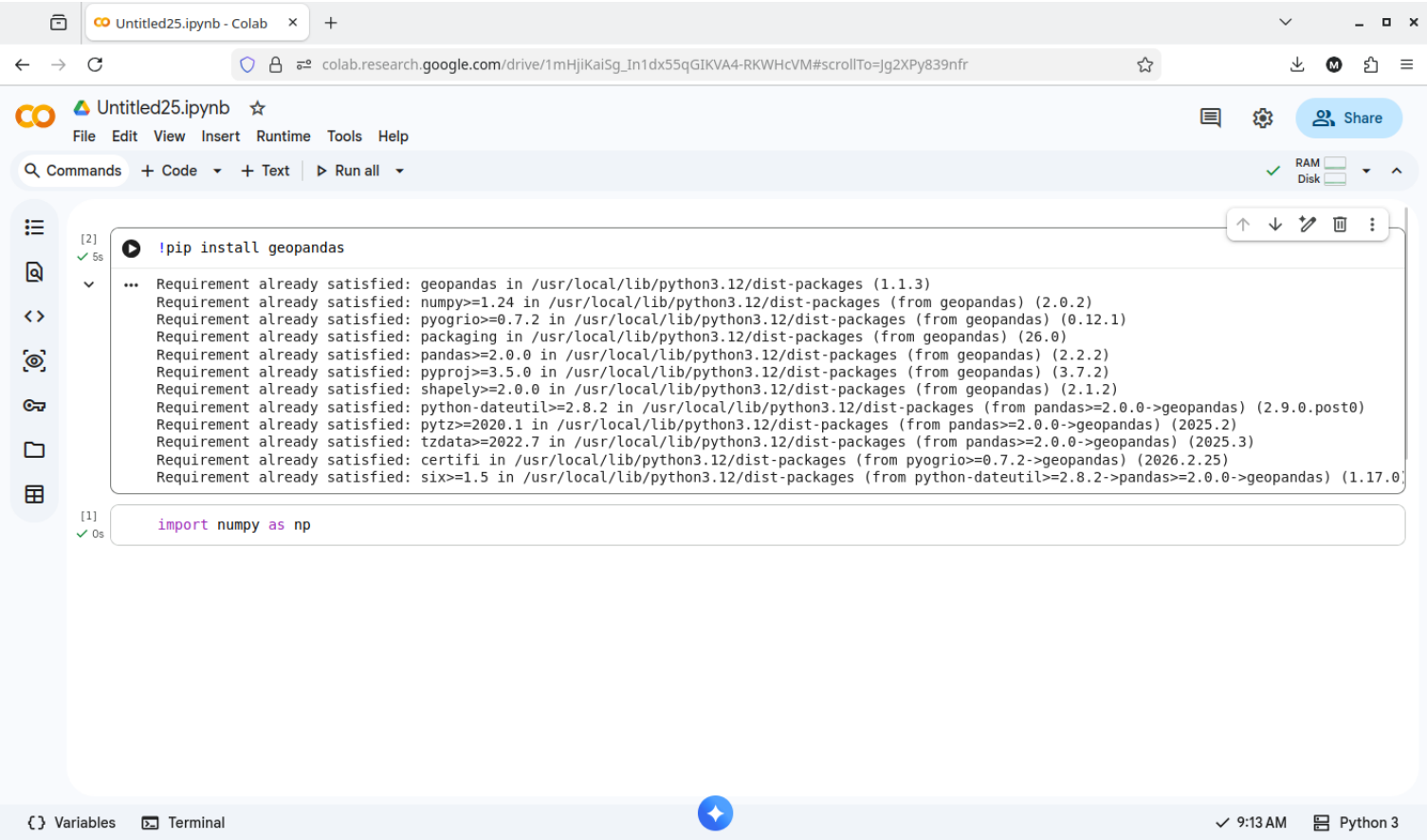
- Não precisa instalar nada
- Roda na nuvem (CPU/GPU)
- Fácil compartilhamento

**Limitações**

- Ambiente temporário
- Dependente de internet

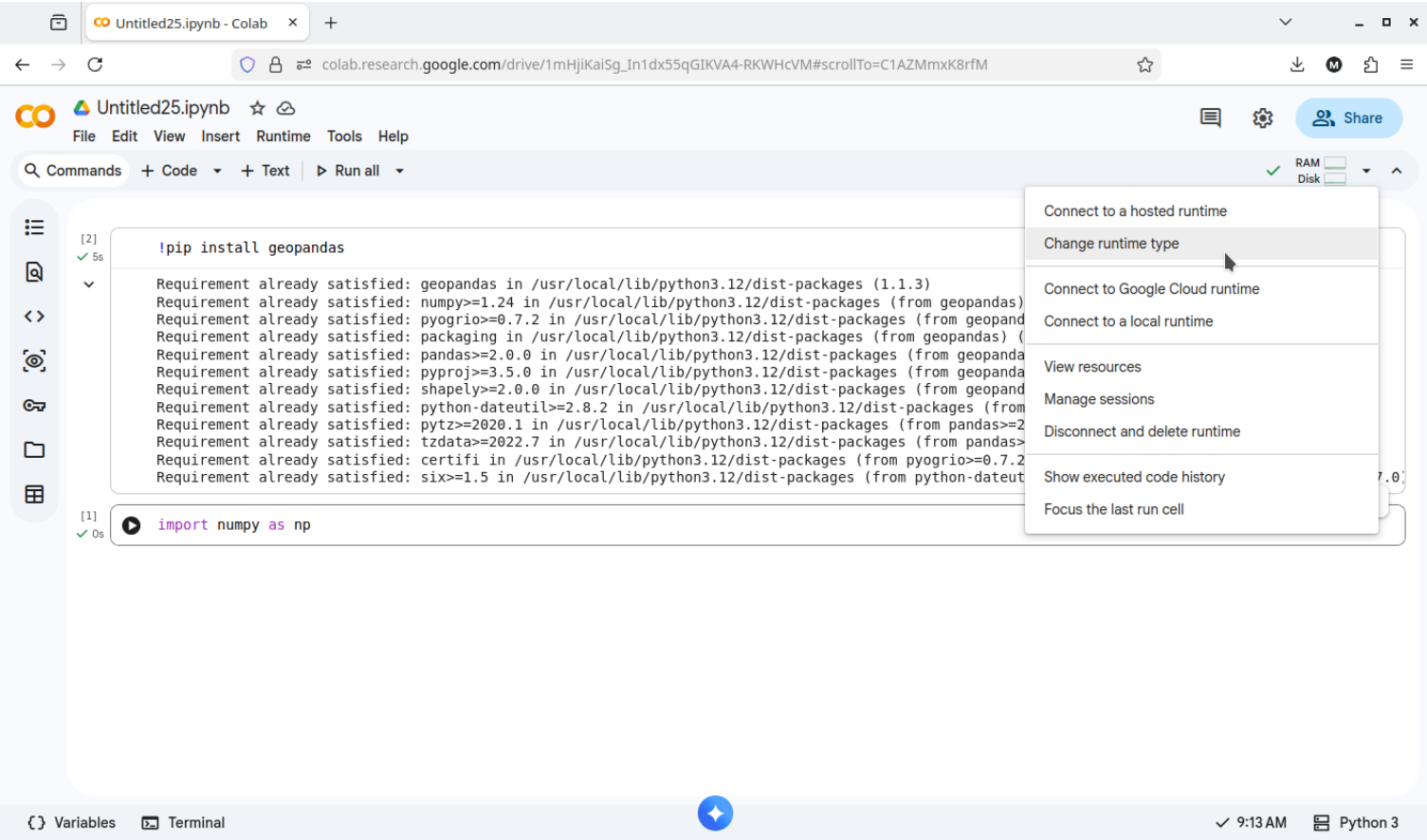
# Google Colab (na nuvem)

- Roda no navegador
- Utiliza uma máquina virtual oferecida pelo Google.
- Pacotes devem ser instalados pelo caderno.



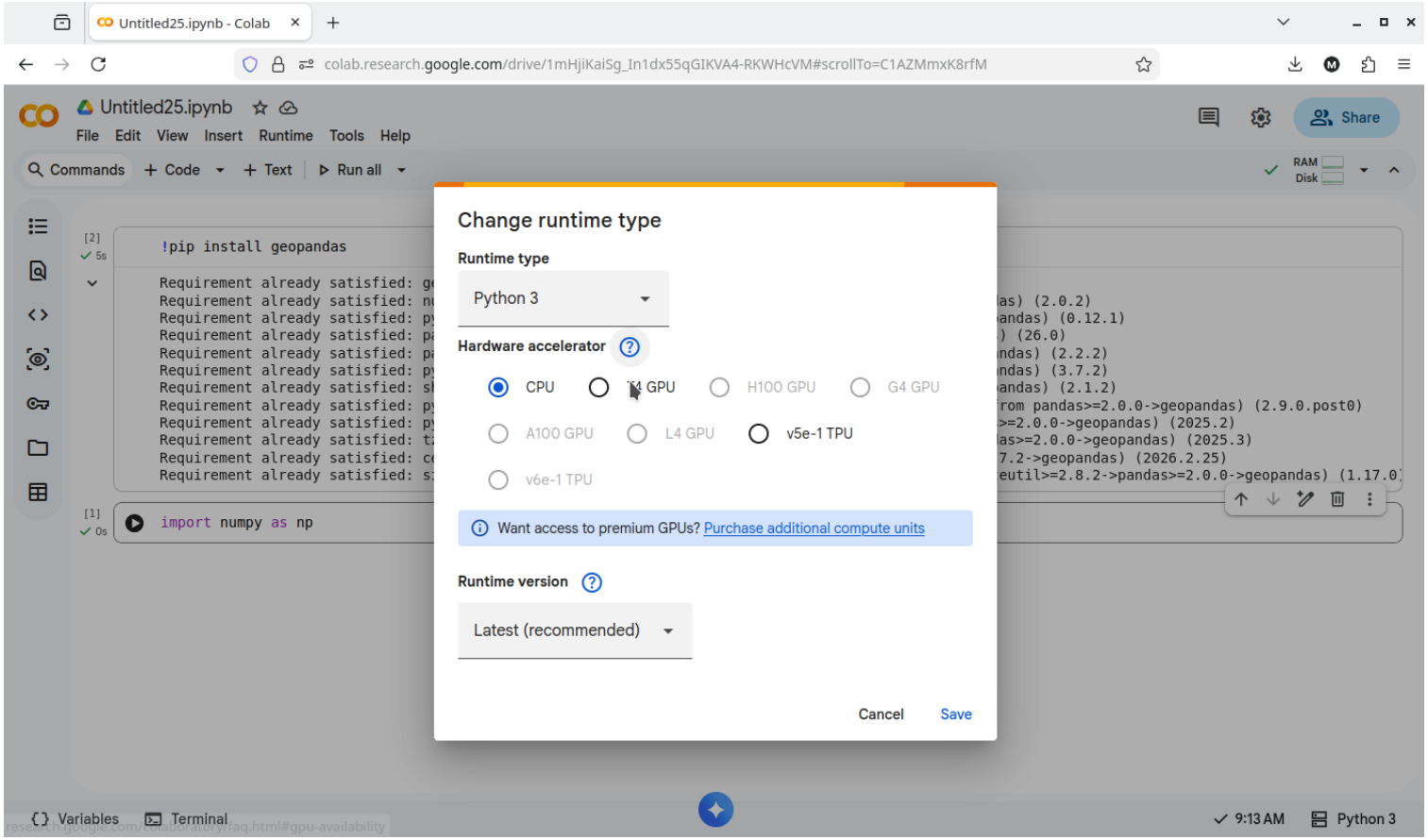
# Google Colab (na nuvem

- Roda no navegador
- Utiliza uma máquina virtual oferecida pelo Google.
- Pacotes devem ser instalados pelo caderno.



# Google Colab (na nuvem)

- Roda no navegador
- Utiliza uma máquina virtual oferecida pelo Google.
- Pacotes devem ser instalados pelo caderno.



# Comandos básicos no Jupyter Notebook

No Jupyter podemos combinar **código**, **comandos especiais** e **texto formatado**.

## Comandos de sistema (!)

Executam comandos do terminal:

```
!ls
!pip install geopandas
!pwd
```

## Magic commands (%)

Comandos especiais do Jupyter:

```
%timeit sum(range(1000))
%matplotlib inline
```

## Markdown

Permite escrever texto formatado.

```
# Título
## Seção

Texto com **negrito**
e *itálico*
```

## Figuras

```
![Mapa](figura.png)
```

## Resumo

- ! → comandos do sistema
- % → comandos do Jupyter
- Markdown → documentação e organização

# JupyterLab (local)

Interface web moderna para notebooks.

**Instalação**

```
pip install jupyterlab
```

```
pixi add jupyterlab
pixi shell
```

**Executar**:

```
jupyter lab
```

**Funcionalidades**

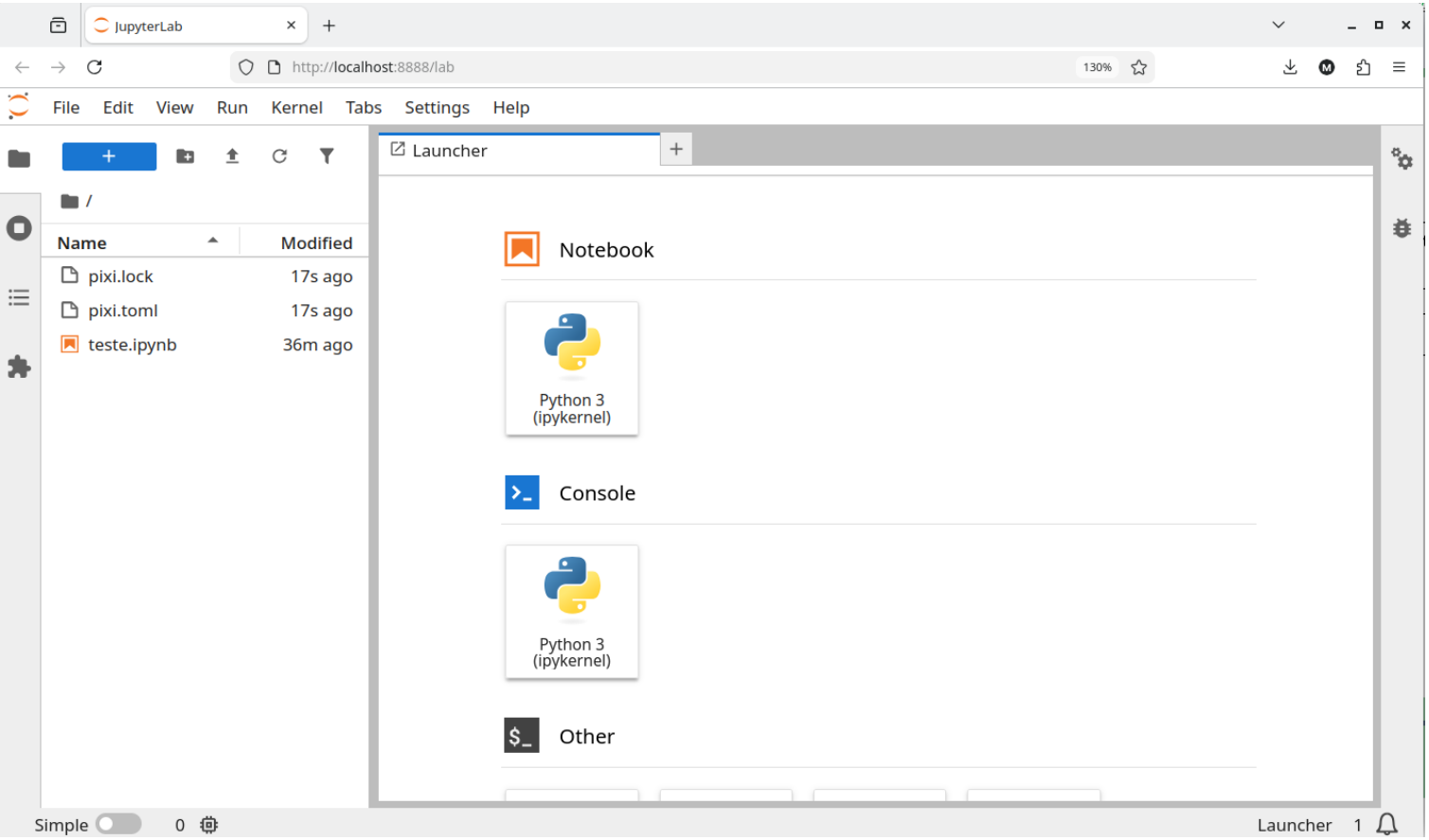
- Interface com abas (tipo IDE)
- Editor de arquivos
- Terminal integrado

**Vantagens**

- Mais poderoso que notebook clássico
- Totalmente local
- Ideal para projetos maiores

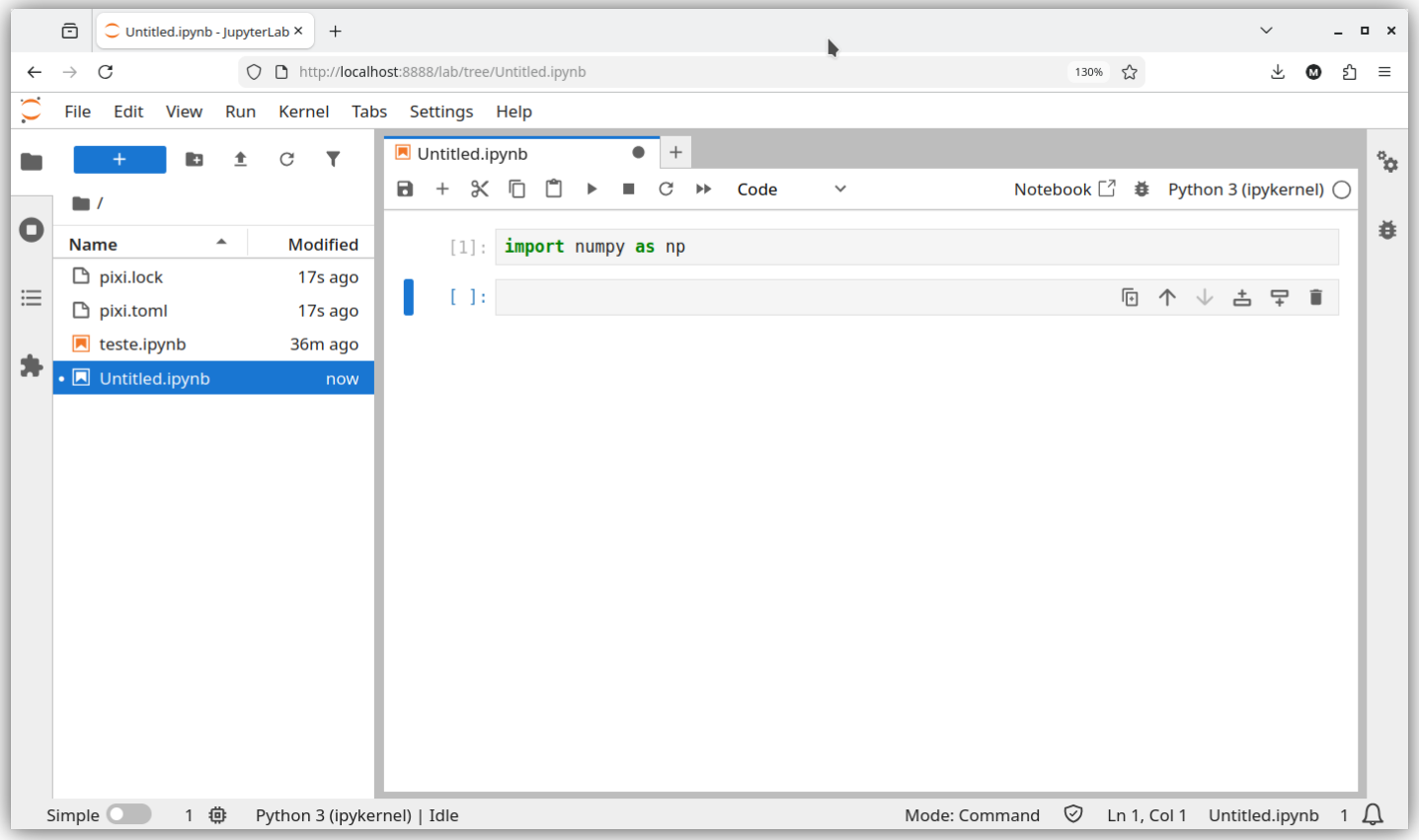
# JupyterLab (local)

- Roda no navegador
- Acessa os pacotes instalados no mesmo ambiente Python.



# JupyterLab (local)

- Roda no navegador
- Acessa os pacotes instalados no mesmo ambiente Python.



# VSCode / VSCodium (plugin Jupyter)

Permite rodar notebooks dentro do editor.

## Instalação

```
# instalar python com pip
pip install notebook ipykernel
```

```
# instalar python com pixi
pixi add notebook ipykernel
```

Instalar extensões:

- Python
- Jupyter

## Uso

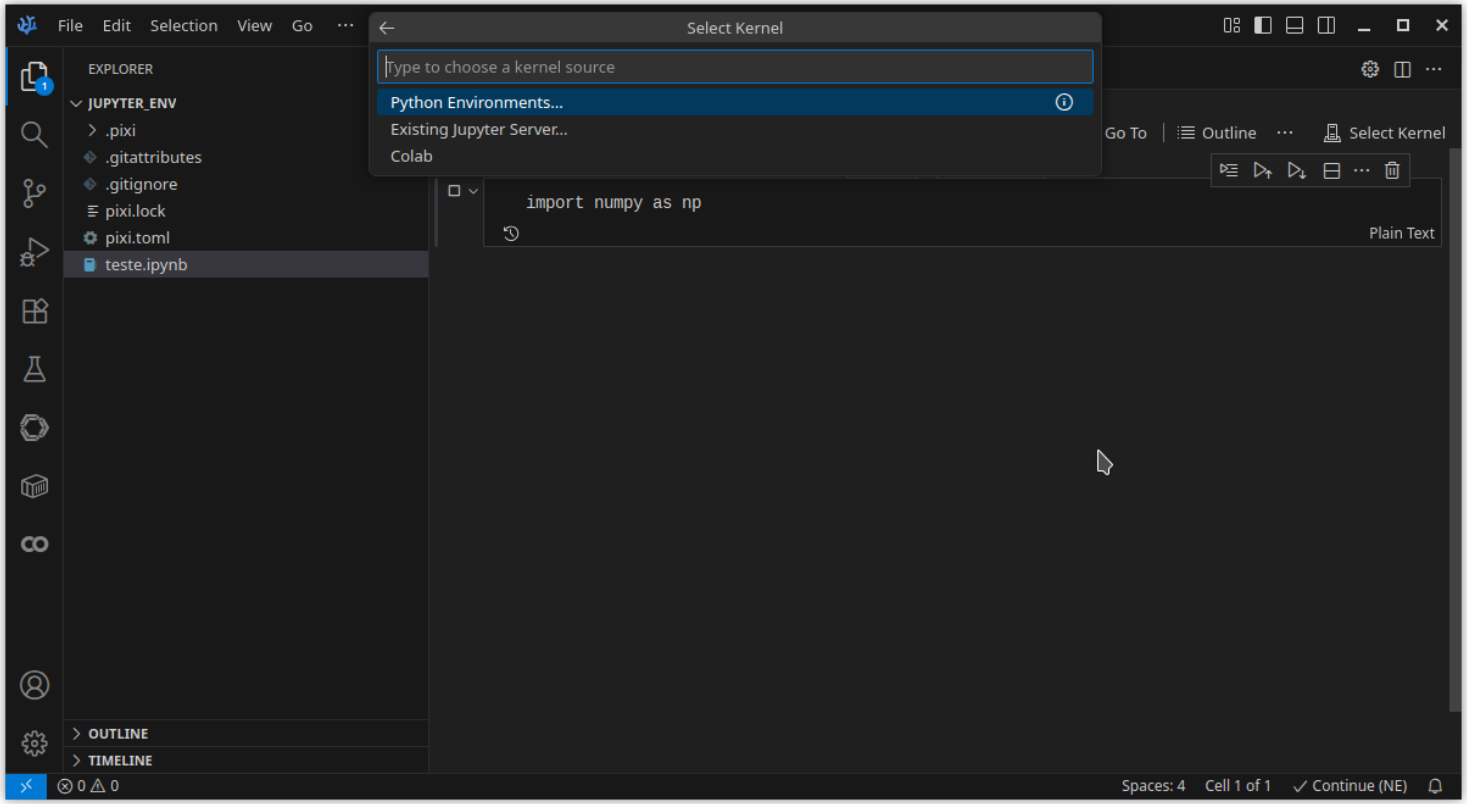
- Abrir ou criar um novo arquivo com extensão `.ipynb`.
- Executar células diretamente

## Vantagens

- Integração com código Python
- Debugging e autocomplete
- Ambiente local controlado

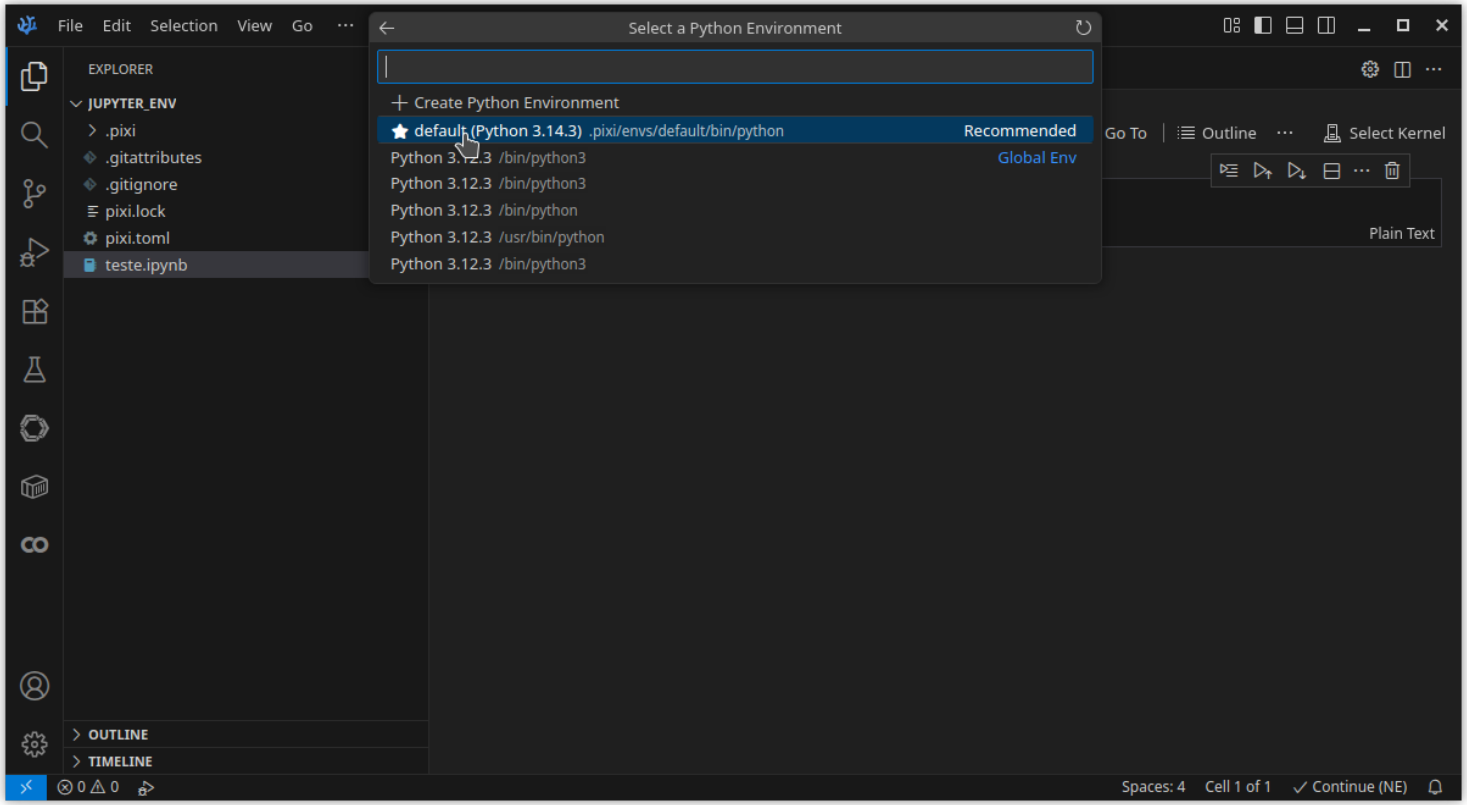
# VSCode / VSCodium (plugin Jupyter)

- Ao executar uma célula, deve-se escolher o ambiente python que executará os comandos.
- Com a extensão **Colab** pode-se escolher um ambiente python **remoto do Google**!



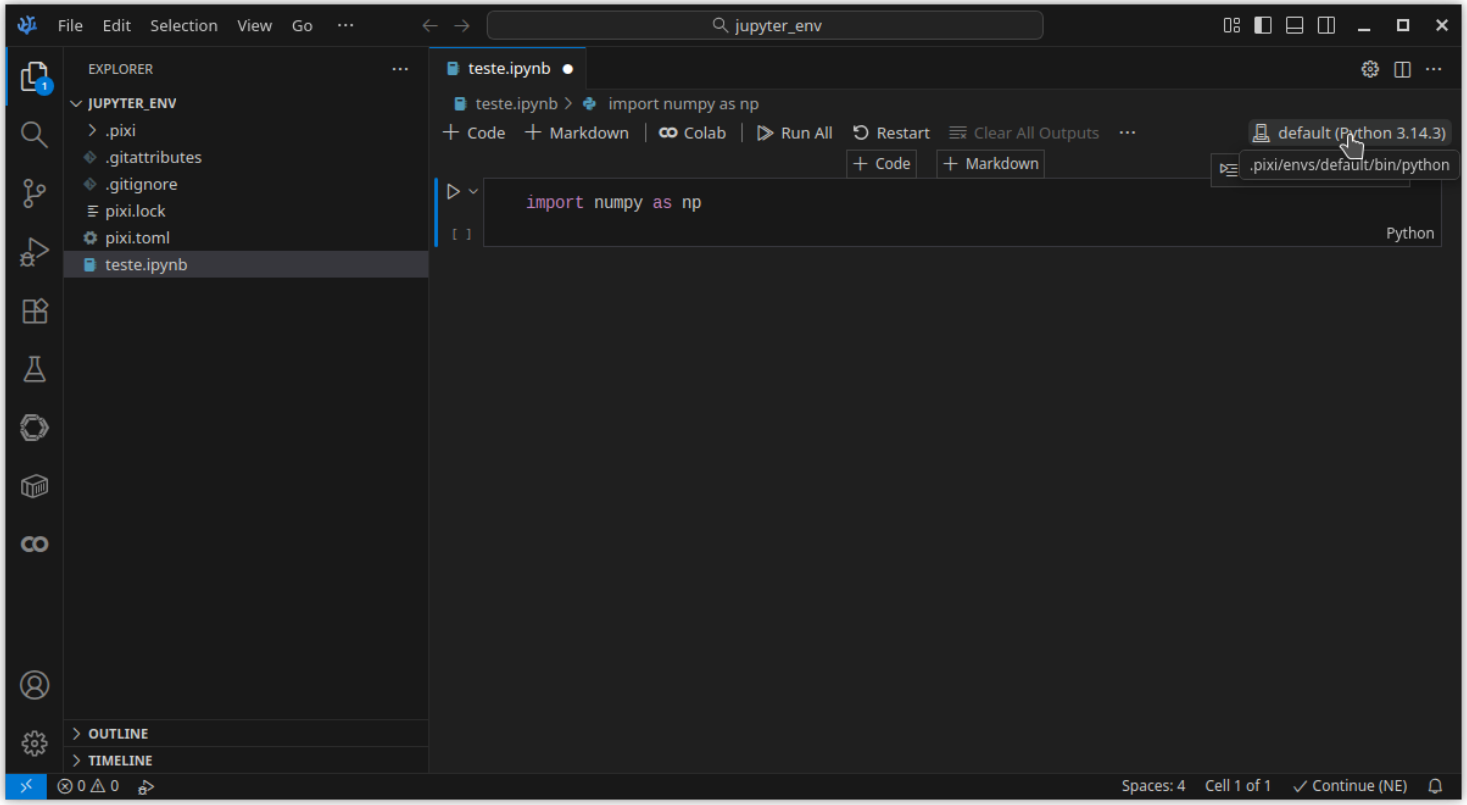
# VSCode / VSCodium (plugin Jupyter)

- Podemos escolher o ambiente python criado com pixi.
- Lembre-se que deve ter os pacotes do jupyter instalados para funcionar como **kernel**.



# VSCode / VSCodium (plugin Jupyter)

- Note o **kernel** em execução no canto superior direito.
- Diferente do local onde observamos o ambiente nos arquivos .py.



# Comparação das abordagens

| Critério | Colab | VSCode | JupyterLab |
|---|---|---|---|
| Instalação | Não | Sim | Sim |
| Offline | Não | Sim | Sim |
| GPU | Sim | Não | Não |
| Facilidade | Alta | Média | Média |
| Controle ambiente | Baixo | Alto | Alto |

**Resumo**

- Iniciantes → Colab
- Desenvolvimento → VSCode
- Projetos maiores → JupyterLab

Cadernos Jupyter com Geopandas e Leafmap

# Estruturas de dados avançadas no Jupyter

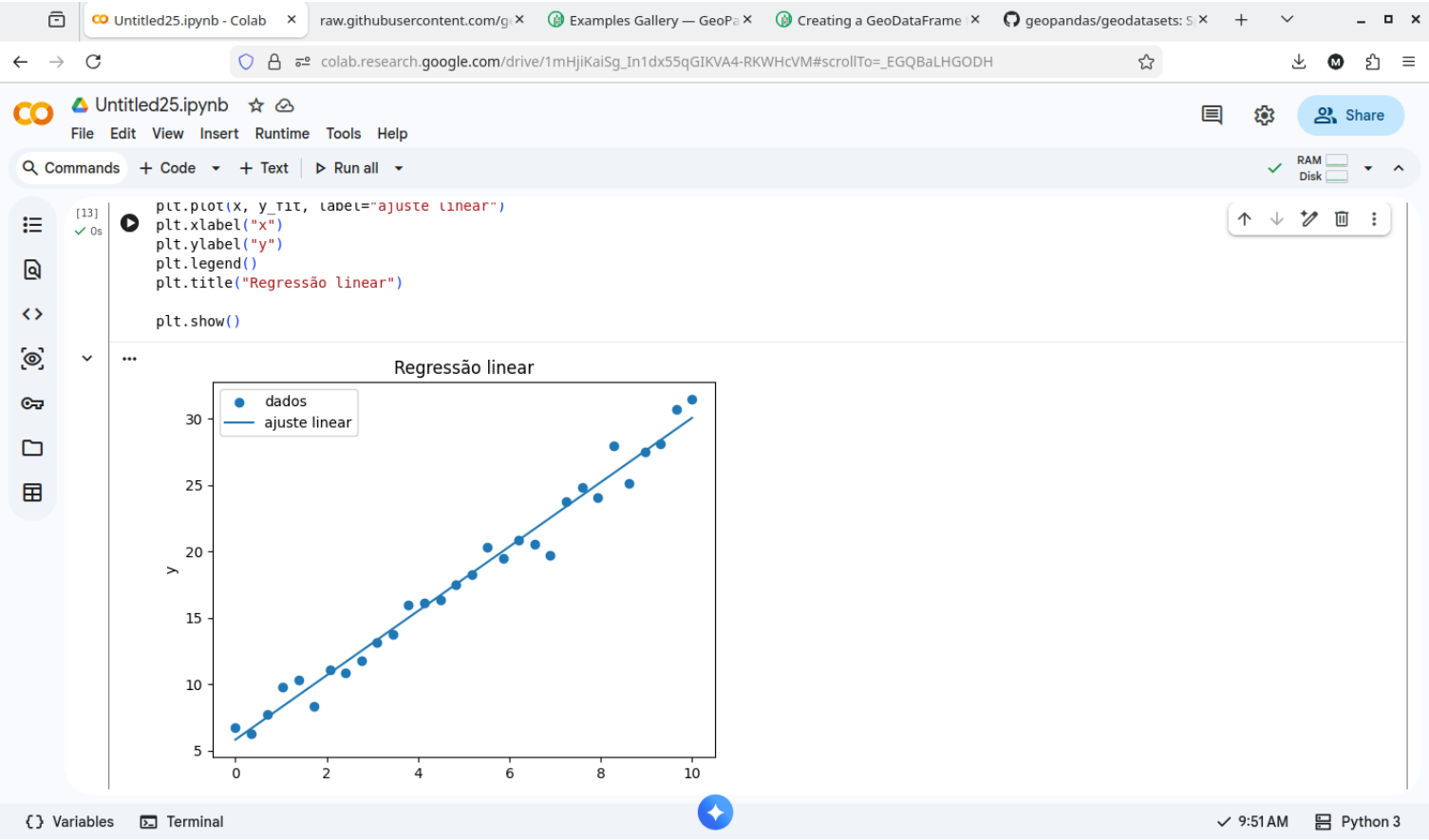
# Matplotlib: Exemplo de regressão linear

Exemplo em um notebook Jupyter com dados sintéticos.

```
import numpy as np
import matplotlib.pyplot as plt

# gerar dados sintéticos
np.random.seed(0)
x = np.linspace(0, 10, 30)
y = 2.5 * x + 5 + np.random.randn(30)

# ajustar regressão linear (y = ax + b)
coef = np.polyfit(x, y, 1)
y_fit = np.polyval(coef, x)

# plot
plt.scatter(x, y, label="dados")
plt.plot(x, y_fit, label="ajuste linear")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.title("Regressão linear")

plt.show()
```

# Matplotlib: Exemplo de regressão linear

- Gráficos da Matplotlib tem visualização como Jupyter Widgets.



# Dataframe no caderno Jupyter.

**Pands/Geopandas** Acesso a *DataFrames*.

## Instalação (Colab)

```
!pip install geopandas
```

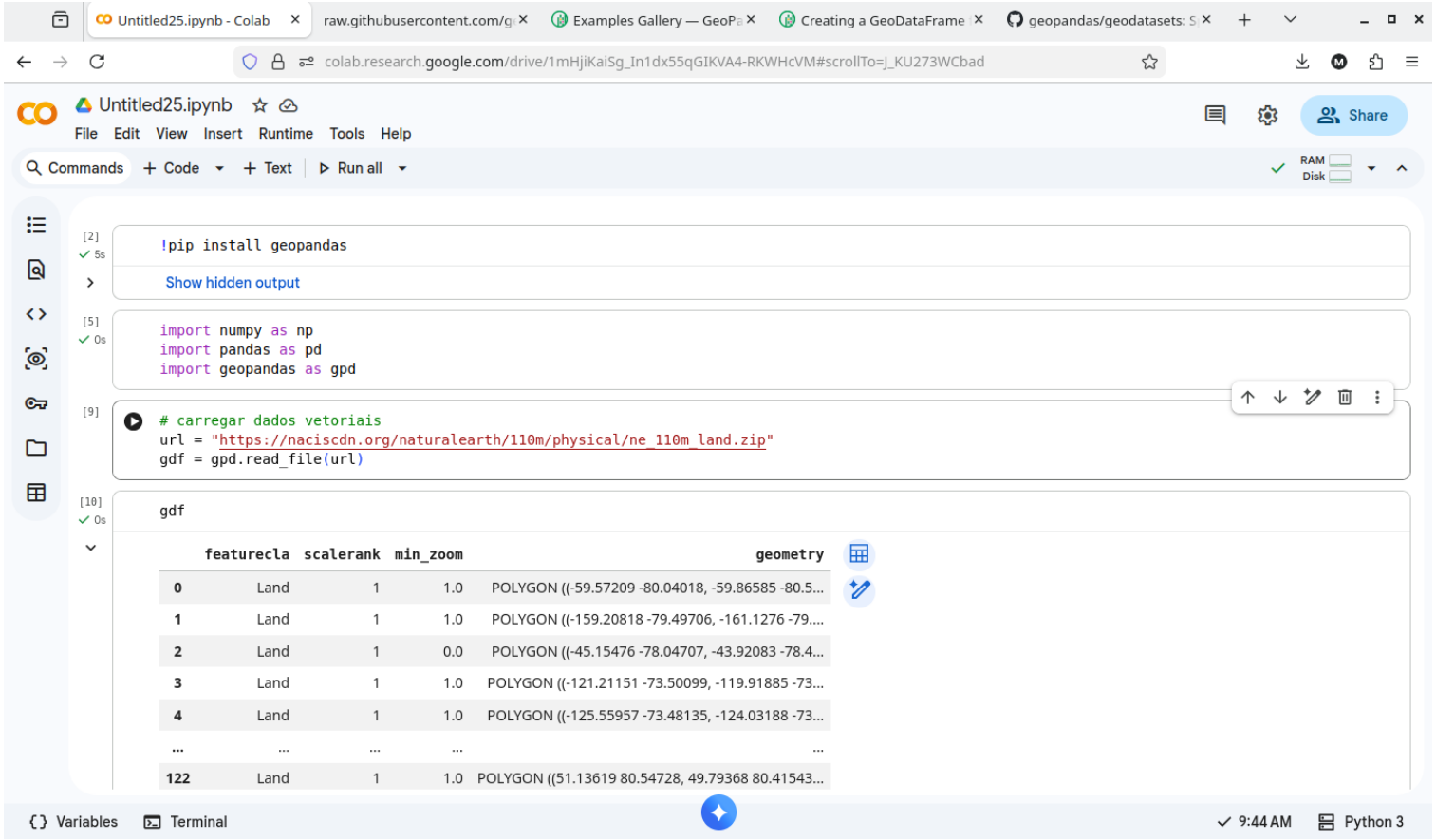
## Código

```
import pandas as pd
import geopandas as gpd
```

```
# carregar dados vetoriais
url = "https://naciscdn.org/naturalearth/110m/physical/ne_110m_land.zip"
gdf = gpd.read_file(url)
gdf
```

# Dataframe no caderno Jupyter.

- Dataframes e geodataframes tem visualização como Jupyter Widgets.



Cadernos Jupyter com Geopandas e Leafmap

# Visualização de geoinformação no Jupyter

# Leafmap em caderno Jupyter

**Contexto:** `leafmap` permite criar mapas interativos usando Python, integrando Jupyter + Leaflet.

**Instalação (Colab)**

```
!pip install leafmap geopandas
```

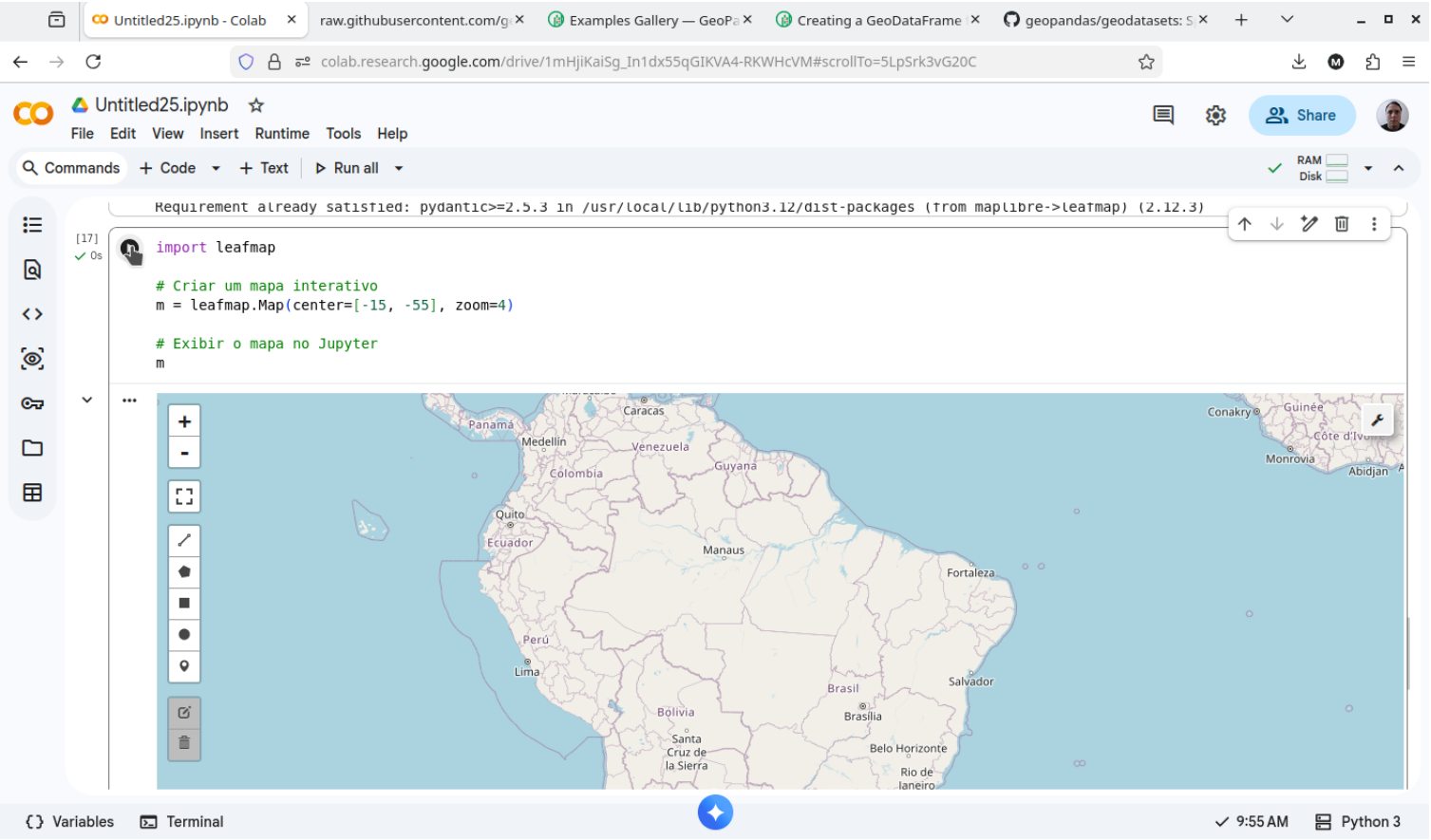
```
import leafmap

# Criar um mapa interativo
m = leafmap.Map(center=[-15, -55], zoom=4)

# Exibir o mapa no Jupyter
m
```

**O que acontece no notebook:**

- O mapa é renderizado interativamente
- Zoom e navegação com o mouse
- Ideal para exploração espacial

# Leafmap em caderno Jupyter

- Leafmap (ou folium, maplibre-gl, etc) apresentam mapas como Jupyter Widgets.

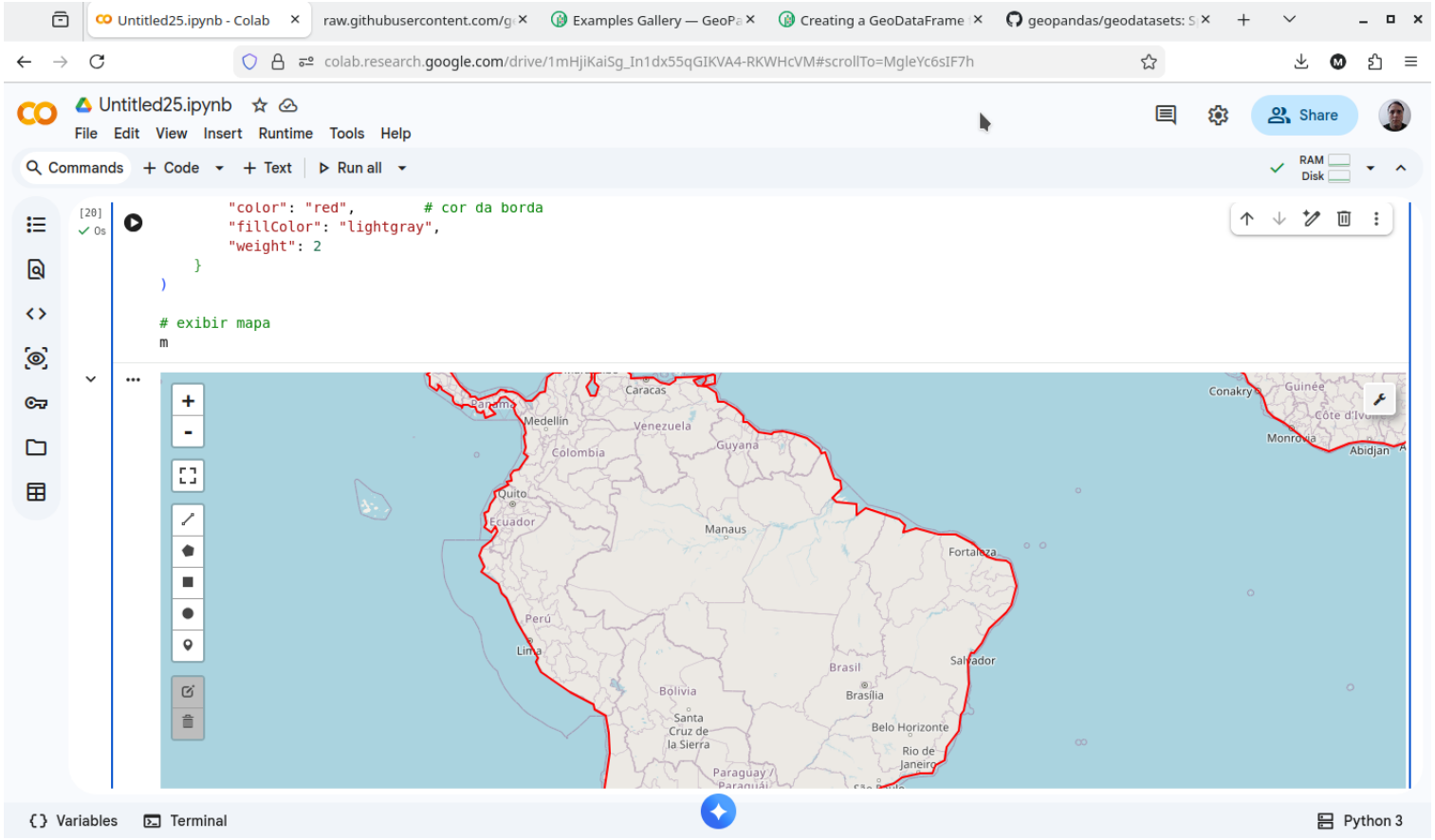


# Leafmap em caderno Jupyter: GeoDataFrame

```
import leafmap
import geopandas as gpd

# criar mapa centrado no Brasil
m = leafmap.Map(center=[-15, -55], zoom=4)

# carregar dados vetoriais
url = "https://naciscdn.org/naturalearth/110m/physical/ne_110m_land.zip"
gdf = gpd.read_file(url)

# adicionar ao mapa com estilo
m.add_gdf(
    gdf,
    layer_name="Natural Earth",
    style={
        "color": "red",         # cor da borda
        "fillColor": "lightgray",
        "weight": 2
    }
)

# exibir mapa
m
```

# Leafmap em caderno Jupyter: GeoDataFrame

- Adicionando uma camada vetorial (GeoDataFrame).



# Leafmap em caderno Jupyter: COG

**COG (Cloud Optimized GeoTIFF)** permite leitura eficiente via HTTP.

## Código

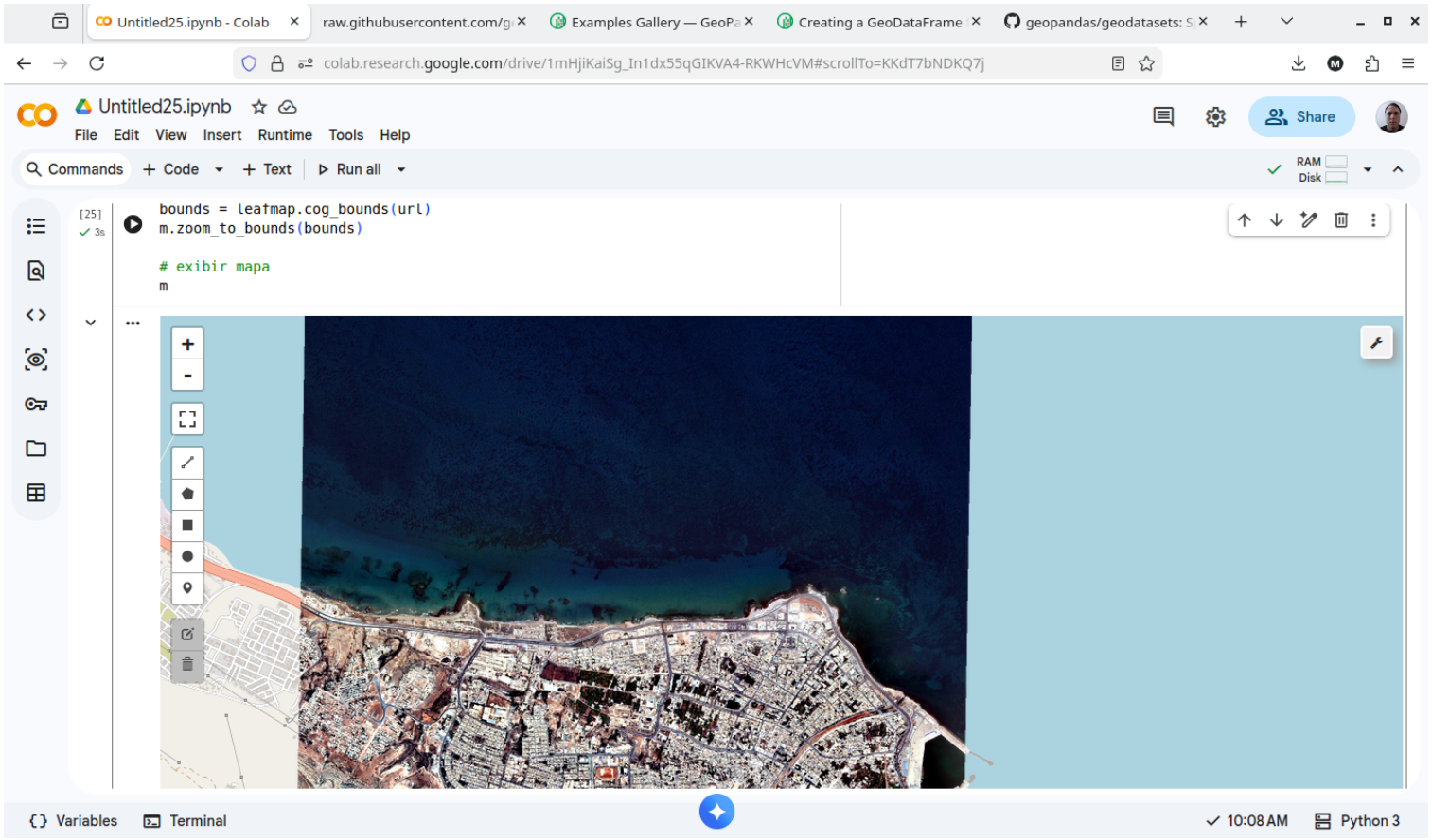
```
import leafmap
m = leafmap.Map(center=[20, 0], zoom=2)

# URL de um COG público (exemplo Sentinel-2)
url = "https://github.com/opengeos/data/releases/download/raster/Libya-2023-07-01.t

# adicionar COG ao mapa
m.add_cog_layer(url, name="COG")

bounds = leafmap.cog_bounds(url)
m.zoom_to_bounds(bounds)
# exibir mapa
m
```

## Vantagens do COG

- Acesso direto via URL (sem download completo)
- Leitura por blocos (otimizado para web)
- Ideal para grandes rasters

# Leafmap em caderno Jupyter: COG

- Adicionando uma camada matricial (COG).



# Referências importantes

DORMAN, Michael et al. Geocomputation with Python. [S.l.]: CRC Press, 2025. Disponível em `https://py.geocompx.org/`.